

TRƯỜNG: ĐH CÔNG THƯƠNG TP.HCM KHOA: CÔNG NGHỆ THÔNG TIN BỘ MÔN: KHOA HỌC DỮ LIỆU MH: Lập trình Python	BUỔI 4: Lập trình hướng đối tượng với Python	
--	---	--

A. MỤC TIÊU

Sau khi học xong Buổi 04 sinh viên có thể:

- Trình bày được các khái niệm cơ bản của lập trình hướng đối tượng gồm lớp (class), đối tượng (object), thuộc tính (attribute) và phương thức (method).
- Xây dựng được một lớp trong Python, khai báo các thuộc tính và sử dụng phương thức `__init__()` để khởi tạo dữ liệu ban đầu cho đối tượng.
- Tạo và sử dụng các đối tượng từ lớp; thực hiện được việc truy cập, cập nhật thuộc tính và gọi các phương thức của đối tượng.
- Phân biệt và sử dụng được thuộc tính đối tượng, thuộc tính lớp; nhận biết và vận dụng các loại phương thức cơ bản như instance method, class method, static method và phương thức đặc biệt.
- Xây dựng được các lớp có thuộc tính và phương thức phù hợp để mô hình hóa và giải quyết một số bài toán cơ bản trong thực tế.
- Vận dụng kiến thức đã học để xây dựng chương trình hướng đối tượng hoàn chỉnh, trong đó các đối tượng có thể lưu trữ dữ liệu, xử lý dữ liệu và thực hiện các hành vi được xác định trong lớp.

B. DỤNG CỤ - THIẾT BỊ THÍ NGHIỆM CHO MỘT SV

STT	Chủng loại – Quy cách vật tư	Số lượng	Đơn vị	Ghi chú
1	Computer	1	1	

C. NỘI DUNG THỰC HÀNH

I. Tóm tắt lý thuyết

1.1. Khái niệm cơ bản

Lập trình hướng đối tượng (**OOP - Object-Oriented Programming**) là mô hình lập trình dựa trên các đối tượng có thuộc tính và hành vi, giúp mô phỏng thế giới thực trong phần mềm. Khác với lập trình thủ tục truyền thống, OOP trong Python được hỗ trợ mạnh mẽ thông qua các khái niệm như lớp (class), đối tượng (object), kế thừa (inheritance), đa hình (polymorphism), và tính đóng gói (encapsulation).

1.1.1. Lớp (class), đối tượng (object), phương thức (method)

Lớp (class) là một mô hình (mẫu thiết kế) trừu tượng dùng để mô tả một nhóm các đối tượng có cùng thuộc tính (dữ liệu) và phương thức (hành vi).

Lớp là một kiểu dữ liệu do người lập trình định nghĩa để tổ chức và quản lý các đối tượng trong chương trình.

Ví dụ: “sinh viên” là một khái niệm chung. Mỗi sinh viên đều có những đặc điểm giống nhau như: họ tên, tuổi, mã số sinh viên, ngành học, và có thể thực hiện các hành động như đăng ký môn học, xem điểm, in thông tin cá nhân... Khái niệm trừu tượng “sinh viên” chính là một lớp (class).

Đặc điểm:

- Class định nghĩa cấu trúc chung, nhưng không lưu trữ dữ liệu cụ thể cho đến khi một đối tượng được tạo.
- Class là nền tảng của lập trình hướng đối tượng, hỗ trợ các nguyên tắc như đóng gói (encapsulation), kế thừa (inheritance), và đa hình (polymorphism).

Đối tượng (object) là một thể hiện cụ thể của một lớp. Mỗi đối tượng có các giá trị riêng biệt (thuộc tính) được định nghĩa trong lớp, và có thể thực hiện các hành vi được mô tả trong lớp.

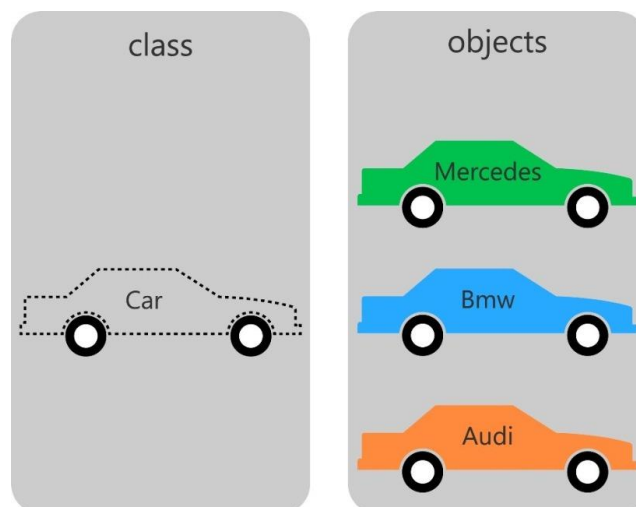
Ví dụ 1.1.

Nếu ĐiệnThoại là lớp chung, thì:

- Iphone_15 là một đối tượng cụ thể của lớp ĐiệnThoại. Iphone_15 có các thuộc tính: Màu sắc = “Đen”, Pin = “4000mAh”
- Xiaomi_14T là một đối tượng cụ thể của lớp ĐiệnThoại. Xiaomi_14T có các thuộc tính: màu sắc = “Xám”, Pin = “5000 mAh”

Đặc điểm:

- Mỗi đối tượng có thể có giá trị riêng cho các thuộc tính của class.
- Nhiều đối tượng có thể được tạo từ cùng một class, mỗi đối tượng độc lập với nhau.



Hình 1.1. Class và object

So sánh class và object:

Bảng 1.1. Bảng so sánh class và object

Lớp (Class)	Đối tượng (Object)
Giống bản thiết kế nhà	Giống ngôi nhà cụ thể xây từ bản thiết kế
Chứa thuộc tính/method	Có giá trị thuộc tính riêng
Ví dụ: class Xe	Ví dụ: xe_toyota = Xe("Xe Toyota", 2020)

Phương thức (method) là một hàm được định nghĩa bên trong lớp, dùng để mô tả hành vi mà các đối tượng của lớp đó có thể thực hiện.

Ví dụ 1.2. HocSinh là lớp mô tả chung về học sinh.

Từ class HocSinh, tạo hai đối tượng:

- Học sinh tên “An” với tuổi 15 và điểm trung bình 8.5
- Học sinh tên “Bình” với tuổi 16 và điểm trung bình 7.0

Các hành vi của học sinh, như phương thức `gioi_thieu()` để in thông tin cá nhân, hoặc `tinh_diem()` để tính toán điểm trung bình.

Đặc điểm:

- Phương thức được gắn với class và chỉ có thể được gọi thông qua đối tượng (hoặc class trong một số trường hợp).
- Khác với hàm thông thường, phương thức trong Python luôn có tham số đầu tiên là **self**, đại diện cho chính đối tượng đang gọi phương thức.
- Phương thức `__init__` là một phương thức đặc biệt, được gọi là phương thức khởi tạo (constructor), dùng để khởi tạo các thuộc tính của đối tượng khi nó được tạo.

Định nghĩa class trong python:

```
class <tên_lớp>([tên_lớp_cha]):  
    [Chuỗi mô tả về lớp -docstring]  
    [Khai báo các thành phần dữ liệu chung của lớp]  
    Định nghĩa phương thức khởi tạo của lớp  
    Định nghĩa các hàm thành phần của lớp
```

Trong đó:

- `<tên_lớp>`: Là một chuỗi biểu diễn tên của một lớp do người dùng đặt theo quy tắc đặt tên trong Python. Chúng ta sẽ tương tác với các đối tượng sinh ra từ lớp bằng cách sử dụng tên này.
- docstrings: mô tả ngắn gọn về lớp, mặc dù là không bắt buộc.

– Các câu lệnh để định nghĩa các thuộc tính và hành vi (phương thức) và phương thức khởi tạo (Constructor).

Ví dụ 1.3.

```
Class Student:
    # "Khởi tạo class Student"
    def __init__(self, name, age, major):
        self.name = name
        self.age = age
        self.major = major
```

Định nghĩa phương thức trong Python:

```
class ClassName:
    def method_name(self, parameters):
        # Method body
        return value # Optional
```

Trong đó:

- def: từ khóa dùng để định nghĩa phương thức.
- tên_phương_thức: tên do người lập trình đặt, theo quy tắc đặt tên hàm.
- self: là tham số bắt buộc đầu tiên của tất cả phương thức, đại diện cho đối tượng gọi phương thức.
- parameters: các tham số, cách nhau bởi dấu phẩy (,)

Ví dụ 1.4.

```
# Định nghĩa class SinhVien
class SinhVien:

    # Phương thức khởi tạo __init__
    def __init__(self, ten, ma_sv, diem_tb):
        self.ten = ten          # Thuộc tính: tên sinh viên
        self.ma_sv = ma_sv      # Thuộc tính: mã sinh viên
        self.diem_tb = diem_tb  # Thuộc tính: điểm trung bình

    # Phương thức: giới thiệu
    def giới_thiệu(self):
        return f'Sinh viên {self.ten}, mã SV: {self.ma_sv}, điểm TB: {self.diem_tb}'

    # Phương thức: kiểm tra học bổng
    def kiểm_tra_hoc_bong(self):
        if self.diem_tb >= 8.5:
            return "Được nhận học bổng"
        return "Không đủ điều kiện học bổng"
```

Để sử dụng phương thức, cần gọi phương thức đó từ một đối tượng hoặc lớp, tùy vào loại phương thức.

Các loại phương thức:

- **Phương thức instance (Instance Methods):** Là phương thức phổ biến nhất, nhận tham số **self** (tham chiếu đến instance của lớp). Có thể truy cập và thay đổi thuộc tính của instance.

- Cú pháp gọi phương thức:

object_name.method_name(arguments)

Ví dụ 1.5.

```
class Dog:
    def __init__(self, name):
        self.name = name

    def bark(self): # Instance method
        return f"{self.name} says Woof!"

dog = Dog("Buddy")
print(dog.bark()) # Output: Buddy says Woof!
```

- **Phương thức lớp (Class Method):** dùng để thao tác với dữ liệu liên quan đến lớp hoặc tạo các phương thức thay thế để khởi tạo đối tượng. Phương thức này nhận tham số đầu tiên là **cls**, đại diện cho lớp và được đánh dấu bằng decorator *@classmethod*.

- Cú pháp gọi phương thức:

ClassName.method_name(arguments)
Hoặc:
object_name.method_name(arguments)

Ví dụ 1.6.

```
class Dog:
    species = "Canis lupus"

    @classmethod
    def get_species(cls): # Class method
        return cls.species

print(Dog.get_species()) # Không tạo thể hiện của lớp
```

- **Phương thức tĩnh (Static Method):** là một phương thức thuộc về lớp (class), không nhận **self** hay **cls** làm tham số đầu tiên, hoạt động như một hàm bình thường nhưng nằm trong lớp và được khai báo bằng decorator *@staticmethod*. Phương thức tĩnh (static method) có thể được gọi thông qua tên lớp mà không cần tạo một thể hiện (instance) của lớp và không thể truy cập thuộc tính của instance hoặc lớp.

- Cú pháp gọi phương thức:

```
ClassName.method_name(arguments)
# Hoặc:
object_name.method_name(arguments)
```

Ví dụ 1.7.

```
class Dog:
    @staticmethod
    def is_pet(): # Static method
        return True

print(Dog.is_pet()) # Output: True
```

– **Phương thức đặc biệt:** Là những phương thức có tên bắt đầu và kết thúc bằng dấu gạch dưới kép (__),

– Python gọi các phương thức này một cách ngầm định để xử lý các thao tác đặc biệt (như khởi tạo, ép kiểu chuỗi, toán tử...).

```
class Student:
    def __init__(self, name):
        self.name = name

    def __str__(self): # Special method
        return f"Student({self.name})"

s = Student("Bob")
print(s) # Output: Student(Bob)
```

1.1.2. Thuộc tính (attribute), hàm khởi tạo __init__

a. Thuộc tính:

Thuộc tính (attribute) trong Python là các biến được gắn với một đối tượng (object) thuộc một lớp (class). Thuộc tính lưu trữ dữ liệu hoặc trạng thái của đối tượng đó. Thuộc tính có thể được truy cập và sử dụng thông qua đối tượng của lớp. Mỗi đối tượng có thể có nhiều thuộc tính để mô tả các đặc điểm riêng biệt của nó.

– Instance attribute (thuộc tính cá thể): Được định nghĩa riêng cho từng đối tượng của lớp. Ví dụ, trong một lớp Sinh Viên, mỗi sinh viên có thể có tên và tuổi khác nhau.

– Class attribute (thuộc tính lớp): Được chia sẻ chung cho tất cả các đối tượng của lớp. Ví dụ, tất cả sinh viên đều học tại một trường đại học cụ thể.

Khai báo:

Instance Attribute: Được khai báo trong phương thức __init__() hoặc phương thức khác bằng từ khóa *self*.

Ví dụ 1.8.

```
class Person:
    def __init__(self, name):
        self.name = name # instance attribute
```

Class Attribute: Khai báo trực tiếp trong thân lớp, bên ngoài các phương thức, không dùng *self*

```
class Person:
    species = "Human" # class attribute
```

Truy cập vào thuộc tính:

- Instance Attribute: truy cập thông qua đối tượng

Ví dụ 1.9.

```
class Person:
    def __init__(self, name):
        self.name = name # instance attribute

p1 = Person("Alice")
p2 = Person("Bob")
print(p1.name) # Alice
print(p2.name) # Bob
```

- Class Attribute: truy cập qua lớp hoặc qua đối tượng (không khuyến khích sử dụng)

Ví dụ 1.10.

```
class Person:
    species = "Human" # class attribute
print(Person.species) # Human

p1 = Person()
p2 = Person()
print(p1.species) # Human
Person.species = "Superhuman"
print(p2.species) # Superhuman (cùng thay đổi)
```

Python không có từ khóa *private*, *public* như Java/C++, nhưng quy ước đặt tên được dùng để thể hiện quyền truy cập.

Bảng 1.2. Bảng quyền truy cập

Tên thuộc tính	Ý nghĩa
name	Public
_name	protected – dùng trong nội bộ/lớp kế thừa

<code>__name</code>	<code>private</code>
---------------------	----------------------

b. Hàm `__init__()`

Hàm `__init__()` là phương thức đặc biệt trong lớp dùng để khởi tạo (initialize) đối tượng. Nó được gọi tự động khi đối tượng được tạo bằng cú pháp `TênLớp()`.

Hàm `__init__()` luôn có tham số ***self*** đầu tiên, sau đó là các tham số khác để truyền giá trị từ bên ngoài vào.

Nếu không định nghĩa `__init__()`, Python sẽ dùng một phiên bản mặc định, không có thuộc tính ban đầu.

Cú pháp:

```
class ClassName:
    def __init__(self, parameters):
        self.attribute = value
```

Trong đó:

- *self* là tham số bắt buộc, đại diện cho chính đối tượng đang được tạo.
- Các tham số còn lại là những giá trị được truyền vào khi khởi tạo đối tượng.

Ví dụ 1.11.

```
class Person:
    def __init__(self, name, age):
        self.name = name      # instance attribute
        self.age = age

# Tạo đối tượng
p1 = Person("Alice", 25)
print(p1.name)  # Alice
print(p1.age)   # 25
# Tạo đối tượng thứ 2
p2 = Person("Bob", 30)
print(p2.name)  # Bob
print(p2.age)   # 30
```

c. Các hàm truy cập vào thuộc tính:

Thông thường để truy cập vào thuộc tính của một đối tượng thông qua toán tử “dấu chấm” (Ví dụ ***player1.name***). Tuy nhiên **Python** cho phép truy cập chúng thông qua hàm (function).

Hàm	Mô tả
<code>getattr(obj, name[, default])</code>	Trả về giá trị của thuộc tính, hoặc trả về giá trị mặc định nếu đối tượng không có thuộc tính này.

hasattr(obj, name)	Kiểm tra xem đối tượng này có thuộc tính cho bởi tham số 'name' hay không.
setattr(obj, name, value)	Set giá trị vào thuộc tính. Nếu thuộc tính không tồn tại, thì nó sẽ được tạo ra.
delattr(obj, name)	Xóa bỏ thuộc tính.

Ví dụ 1.12:

```

player1 = Player("Tom", 20)

# getattr(obj, name[, default])
print ("getattr(player1, 'name') = " , getattr(player1, "name") )

print ("setattr(player1, 'age', 21): ")

# setattr(obj, name, value)
setattr(player1, "age", 21)

print ("player1.age = ", player1.age)

# Kiểm tra player1 có thuộc tính (attribute) address hay không?
hasAddress = hasattr(player1, "address")

print ("hasattr(player1, 'address') ? ", hasAddress)

# Tạo thuộc tính 'address' cho đối tượng 'player1'.
print ("Create attribute 'address' for object 'player1'")
setattr(player1, 'address', "USA")

print ("player1.address = ", player1.address)

# Xóa thuộc tính 'address'.
delattr(player1, "address")

```

```

-----Sử dụng hàm truy cập thuộc tính-----
getattr(player1, 'name') = Tom
setattr(player1, 'age', 21):
player1.age = 21
hasattr(player1, 'address') ? False
Create attribute 'address' for object 'player1'
player1.address = USA

```

Các thuộc tính có sẵn của class:

Các lớp của **Python** đều là hậu duệ của lớp **object**. Và vì vậy nó thừa kế các thuộc tính sau:

Thuộc tính	Mô tả
__dict__	Đưa ra thông tin về lớp này một cách ngắn gọn, dễ hiểu, như một bộ từ điển (Dictionary)

<code>__doc__</code>	Trả về chuỗi mô tả về class, hoặc trả về None nếu nó không được định nghĩa
<code>__class__</code>	Trả về một đối tượng, chứa thông tin về lớp, đối tượng này có nhiều thuộc tính có ích, trong đó có thuộc tính <code>__name__</code> .
<code>__module__</code>	Trả về tên module của lớp, hoặc trả về “ <code>__main__</code> ” nếu lớp đó được định nghĩa trong module đang được chạy.

Ví dụ 1.13:

```
#Các thuộc tính có sẵn của class:
class Customer :
    'This is Customer class'

    def __init__(self, name, phone, address):

        self.name = name
        self.phone = phone
        self.address = address

john = Customer("John",1234567, "USA")

print ("")
print ("Các thuộc tính có sẵn của class")
print ("john.__dict__ = ", john.__dict__)

print ("john.__doc__ = ", john.__doc__)

print ("john.__class__ = ", john.__class__)
print ("john.__class__.__name__ = ", john.__class__.__name__)
print ("john.__module__ = ", john.__module__)
```

Kết quả:

```
Các thuộc tính có sẵn của class
john.__dict__ = {'name': 'John', 'phone': 1234567, 'address': 'USA'}
john.__doc__ = This is Customer class
john.__class__ = <class '__main__.Customer'>
john.__class__.__name__ = Customer
john.__module__ = __main__
```

Biến của lớp:

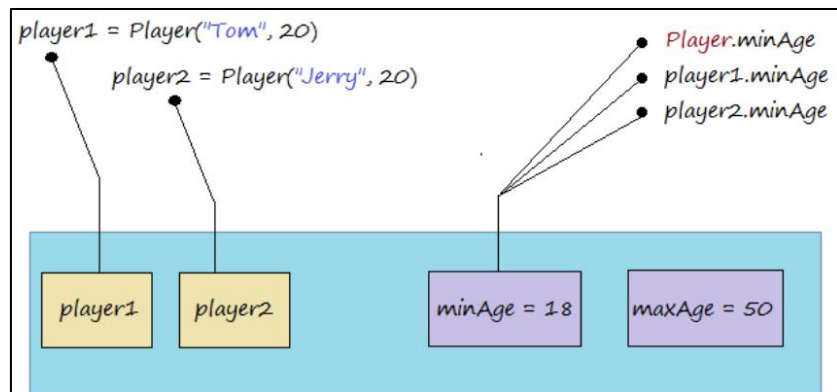
Trong **Python** khái niệm “Biến của lớp (Class's Variable)” tương đương với khái niệm trường tĩnh (Static Field) của các ngôn ngữ khác như **Java**, **CSharp**. Biến của lớp có thể được truy cập thông qua tên lớp hoặc thông qua đối tượng.

```
class Player:
    # Biến của Lớp.
    minAge = 18
    maxAge = 50

    def __init__(self, name, age):
        self.name = name
        self.age = age
```

Chú ý: chúng ta nên truy cập “biến của lớp” thông qua tên lớp thay vì truy cập thông qua đối tượng. Điều này giúp tránh nhầm lẫn giữa “biến của lớp” và thuộc tính.

Ví dụ 1.14



```
#Biến của Lớp
player1 = Player("Tom", 20)
player2 = Player("Jerry", 20)
print("")
print ("Biến của lớp")
# Truy cập thông qua tên Lớp.
print ("Player.minAge = ", Player.minAge)

# Truy cập thông qua đối tượng.
print ("player1.minAge = ", player1.minAge)
print ("player2.minAge = ", player2.minAge)
print (" ----- ")
print ("Assign new value to minAge via class name, and print..")

# Gán một giá trị mới cho minAge thông qua tên Lớp.
Player.minAge = 19

print ("Player.minAge = ", Player.minAge)
print ("player1.minAge = ", player1.minAge)
print ("player2.minAge = ", player2.minAge)
```

Kết quả:

```
Biến của lớp
Player.minAge = 18
player1.minAge = 18
player2.minAge = 18
-----
Assign new value to minAge via class name, and print..
Player.minAge = 19
player1.minAge = 19
player2.minAge = 19
```

Liệt kê danh sách các thành viên của lớp hoặc đối tượng:

Python cung cấp cho bạn hàm **dir**, hàm này liệt kê ra danh sách các phương thức, thuộc tính, biến của lớp hoặc của đối tượng.

Ví dụ 1.15

```
#Liệt kê danh sách các thành viên của lớp hoặc đối tượng

print ("" )
print ("Liệt kê danh sách các thành viên của lớp hoặc đối tượng")
print ( dir(Player) )
print ("\n\n")
player1 = Player("Tom", 20)
player1.address = "USA"

# In ra danh sách các thuộc tính, phương thức và biến của đối tượng 'player1'.
print ( dir(player1) )
```

Kết quả:

```
Liệt kê danh sách các thành viên của lớp hoặc đối tượng
['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattr__', '__gt__',
 '__hash__', '__init__', '__init_subclass__', '__le__', '__lt__', '__module__', '__ne__', '__new__', '__reduce__',
 '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', '__weakref__', 'maxAge', 'minAge']

['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattr__', '__gt__',
 '__hash__', '__init__', '__init_subclass__', '__le__', '__lt__', '__module__', '__ne__', '__new__', '__reduce__',
 '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', '__weakref__', 'address', 'age',
 'maxAge', 'minAge', 'name']
```

II. BÀI TẬP HƯỚNG DẪN MẪU

Bài 1. Tạo lớp xe (Car) và xây dựng các thuộc tính cho lớp Car: loại xe và các thuộc tính đối tượng: tên xe, màu sắc, nguyên liệu. (Đặt tên file HDM_BaiTap01.py)

```
class Car:

    # thuộc tính lớp
    loaixe = "Ô tô"

    # thuộc tính đối tượng
    def __init__(self, tenxe, mausac, nguyenvlieu):
        self.tenxe = tenxe
        self.mausac = mausac
        self.nguyenvlieu = nguyenvlieu

# instantiate the Car class
toyota = Car("Toyota", "Đỏ", "Điện")
lamborghini = Car("Lamborghini", "Vàng", "Deisel")
porsche = Car("Porsche", "Xanh", "Gas")

# access the class attributes
print("Porsche là {}".format(porsche.__class__.loaixe))
print("Toyota là {}".format(toyota.__class__.loaixe))
print("Lamborghini cũng là {}".format(lamborghini.__class__.loaixe))
```

```
# access the instance attributes
print("Xe {} có màu {}. {} là nguyên liệu vận hành.".format(toyota.tenxe, toyota.mausac,
toyota.nguyenlieu))
print("Xe {} có màu {}. {} là nguyên liệu vận hành.".format(lamborghini.tenxe,
lamborghini.mausac,lamborghini.nguyenlieu))
print("Xe {} có màu {}. {} là nguyên liệu vận hành.".format(porsche.tenxe,
porsche.mausac, porsche.nguyenlieu))
```

Kết quả:

```
Porsche là Ô tô.
Toyota là Ô tô.
Lamborghini cũng là Ô tô.
Xe Toyota có màu Đỏ. Điện là nguyên liệu vận hành.
Xe Lamborghini có màu Vàng. Deisel là nguyên liệu vận hành.
Xe Porsche có màu Xanh. Gas là nguyên liệu vận hành.
```

Chương trình trên tạo một lớp *Car*, sau đó xác định các thuộc tính, đặc điểm của đối tượng

Chúng ta truy cập thuộc tính class bằng cách sử dụng `__class__.loaixie`. Các thuộc tính lớp được chia sẻ cho tất cả các cá thể của lớp.

Tương tự, chúng ta truy cập các thuộc tính instance bằng cách sử dụng `toyota.tenxe`, `toyota.mausac` và `toyota.nguyenlieu`.

```
class Car:

    # thuộc tính đối tượng
    def __init__(self, tenxe, mausac, nguyenlieu):
        self.tenxe = tenxe
        self.mausac = mausac
        self.nguyenlieu = nguyenlieu

    # phương thức
    def dungxe(self, mucdich):
        return "{} đang dừng xe để {}".format(self.tenxe,mucdich)

    def chayxe(self):
        return "{} đang chạy trên đường".format(self.tenxe)

    def nomay(self):
        return "{} đang nổ máy".format(self.tenxe)

# instantiate the Car class
```

```
toyota = Car("Toyota", "Đỏ", "Điện")
lamborghini = Car("Lamborghini", "Vàng", "Deisel")
porsche = Car("Porsche", "Xanh", "Gas")

# call our instance methods
print(toyota.dungxe("nạp điện"))
print(lamborghini.chayxe())
print(porsche.nomay())
```

Kết quả:

```
Toyota đang dừng xe để nạp điện
Lamborghini đang chạy trên đường
Porsche đang nổ máy
```

Bài 2. Tạo một lớp một lớp thực hiện thao tác **Máy tính** tay bao gồm các phép toán +, -, *, / (Đặt tên file HDM_BaiTap02.py)

Yêu cầu:

1. Tạo một lớp và sử dụng hàm để khởi tạo các giá trị của lớp đó.
2. Tạo các phương thức cộng, trừ, nhân, chia hai số và trả về kết quả tương ứng.
3. Lấy hai số làm đầu vào và tạo một đối tượng cho lớp truyền hai số đó làm tham số cho lớp.
4. Sử dụng đối tượng, gọi hàm tương ứng tùy theo lựa chọn của người dùng.
5. In kết quả cuối cùng.
6. Thoát

```
class calculator_implementation():
    def __init__(self,in_1,in_2):
        self.a=in_1
        self.b=in_2
    def add_vals(self):
        return self.a+self.b
    def multiply_vals(self):
        return self.a*self.b
    def divide_vals(self):
        return self.a/self.b
    def subtract_vals(self):
        return self.a-self.b
input_1 = int(input("Enter the first number: "))
input_2 = int(input("Enter the second number: "))
print("The entered first and second numbers are : ")
print(input_1, input_2)
my_instance = calculator_implementation(input_1,input_2)
choice=1
```

```

while choice!=0:
    print("0. Exit")
    print("1. Addition")
    print("2. Subtraction")
    print("3. Multiplication")
    print("4. Division")
    choice=int(input("Enter your choice... "))
    if choice==1:
        print("The computed addition result is : ",my_instance.add_vals())
    elif choice==2:
        print("The computed subtraction result is : ",my_instance.subtract_vals())
    elif choice==3:
        print("The computed product result is : ",my_instance.multiply_vals())
    elif choice==4:
        print("The computed division result is : ",round(my_instance.divide_vals(),2))
    elif choice==0:
        print("Exit")
    else:
        print("Sorry, invalid choice!")
print()

```

Bài 3. Xây dựng một chương trình lấy chiều dài và chiều rộng từ người dùng và tìm diện tích hình chữ nhật bằng cách sử dụng các lớp (**Đặt tên file HDM_BaiTap03.py**)

Yêu cầu:

1. Lấy giá trị chiều dài và chiều rộng từ người dùng.
2. Tạo một lớp và sử dụng hàm tạo để khởi tạo các giá trị của lớp đó.
3. Tạo một phương thức gọi là diện tích và trả về diện tích của lớp.
4. Tạo đối tượng cho lớp.
5. Sử dụng đối tượng, gọi phương thức vùng() với các tham số là chiều dài và chiều rộng được lấy từ người dùng.
6. In kết quả
7. Thoát

```

class rectangle():
    def __init__(self,breadth,length):
        self.breadth=breathd
        self.length=length
    def area(self):
        return self.breadth*self.length
a=int(input("Enter length of rectangle: "))
b=int(input("Enter breadth of rectangle: "))
obj=rectangle(a,b)
print("Area of rectangle:",obj.area())

print()

```

III. BÀI TẬP TRÊN LỚP

A. Phần Cơ bản

Bài 1. Lớp Hình tròn

Xây dựng lớp **Circle** dùng để biểu diễn một hình tròn.

Thuộc tính:

- radius: lưu bán kính của hình tròn.

Các phương thức:

1. `__init__(self, radius)`
 - Là phương thức khởi tạo đối tượng hình tròn.
2. `perimeter(self)`
 - Dùng để tính chu vi của hình tròn.
3. `area(self)`
 - Dùng để tính diện tích hình tròn.
4. `display(self)`
 - Dùng để hiển thị thông tin của đối tượng: hiển thị bán kính, chu vi và diện tích của hình tròn.

Yêu cầu chương trình chính:

- Tạo đối tượng **Circle**.
- Gọi phương thức `display()` để hiển thị kết quả.

Bài 2. Lớp Hình chữ nhật

Xây dựng lớp **Rectangle** biểu diễn hình chữ nhật.

Thuộc tính:

- length: chiều dài.
- width: chiều rộng.

Các phương thức:

1. `__init__(self, length, width)`
 - Khởi tạo một đối tượng hình chữ nhật.
 - Nhận chiều dài và chiều rộng rồi lưu vào đối tượng.
2. `perimeter(self)`
 - Tính chu vi hình chữ nhật.
3. `area(self)`
 - Tính diện tích hình chữ nhật.
4. `is_square(self)`
 - Kiểm tra hình chữ nhật hiện tại có phải là hình vuông hay không.
5. `display(self)`
 - Hiển thị chiều dài, chiều rộng, chu vi và diện tích.
 - Đồng thời cho biết đối tượng có phải hình vuông hay không.

Yêu cầu:

Tạo ít nhất hai đối tượng **Rectangle** và hiển thị thông tin từng đối tượng.

Bài 3. Lớp Sinh viên

Xây dựng lớp Student.

Thuộc tính:

- `student_id`: mã sinh viên.
- `full_name`: họ tên.
- `age`: tuổi.
- `gpa`: điểm trung bình.

Các phương thức:

1. `__init__(self, student_id, full_name, age, gpa)`
 - Khởi tạo thông tin ban đầu cho một sinh viên.
2. `input_info(self)`
 - Dùng để nhập thông tin của sinh viên từ bàn phím.
 - Kiểm tra mã và họ tên không được rỗng.
 - Kiểm tra tuổi phải từ 18 trở lên.
 - Kiểm tra điểm trung bình thuộc đoạn từ 0 đến 10.
3. `is_scholarship(self)`
 - Kiểm tra sinh viên có đủ điều kiện nhận học bổng hay không.
 - Nếu điểm trung bình từ 8.0 trở lên thì sinh viên đủ điều kiện.
4. `display(self)`
 - Hiển thị đầy đủ thông tin sinh viên.
 - Hiển thị mã, họ tên, tuổi, điểm trung bình.
 - Gọi `is_scholarship()` để cho biết sinh viên được hay không được học bổng.

Bài 4. Lớp Nhân viên

Xây dựng lớp Employee.

Thuộc tính:

- `employee_id`: mã nhân viên.
- `full_name`: họ tên.
- `base_salary`: lương cơ bản.
- `working_days`: số ngày công.

Các phương thức:

1. `__init__(self, employee_id, full_name, base_salary, working_days)`
 - Khởi tạo thông tin của một nhân viên.
2. `calculate_salary(self)`
 - Tính lương thực nhận của nhân viên.
 - Công thức: $\text{Lương thực nhận} = \text{lương cơ bản} \times \text{số ngày công} / 26$
 - Trả về số tiền lương thực nhận.
3. `display(self)`
 - Hiển thị mã nhân viên, họ tên, lương cơ bản và số ngày công.
 - Gọi `calculate_salary()` để hiển thị lương thực nhận.

Bài 5. Lớp Sản phẩm

Xây dựng lớp Product.

Thuộc tính:

- product_id: mã sản phẩm.
- product_name: tên sản phẩm.
- price: đơn giá.
- quantity: số lượng.

Các phương thức:

1. `__init__(self, product_id, product_name, price, quantity)`
 - Khởi tạo thông tin của sản phẩm.
2. `amount(self)`
 - Tính thành tiền trước giảm giá.
 - Công thức: $\text{price} \times \text{quantity}$.
3. `discount(self)`
 - Tính số tiền được giảm.
 - Nếu số lượng từ 10 trở lên thì giảm 5% thành tiền.
 - Nếu số lượng nhỏ hơn 10 thì tiền giảm bằng 0.
4. `payment(self)`
 - Tính số tiền khách hàng thực tế phải trả.
 - Công thức:
$$\text{Số tiền phải trả} = \text{thành tiền} - \text{tiền giảm}$$
 - Trả về số tiền cuối cùng.
5. `display(self)`
 - Hiển thị thông tin sản phẩm.
 - Hiển thị mã, tên, đơn giá, số lượng, thành tiền, tiền giảm và số tiền phải trả.

Bài 6. Lớp Tài khoản ngân hàng

Xây dựng lớp BankAccount.

Thuộc tính:

- account_number: số tài khoản.
- owner_name: tên chủ tài khoản.
- balance: số dư.

Các phương thức:

1. `__init__(self, account_number, owner_name, balance)`
 - Khởi tạo thông tin cho tài khoản ngân hàng.
2. `deposit(self, amount)`
 - Thực hiện giao dịch gửi tiền vào tài khoản.
 - Kiểm tra số tiền gửi phải lớn hơn 0.
 - Nếu hợp lệ, cộng số tiền vào balance.
3. `withdraw(self, amount)`

- Thực hiện giao dịch rút tiền.
 - Kiểm tra số tiền rút phải lớn hơn 0.
 - Không cho phép rút số tiền lớn hơn số dư.
 - Nếu hợp lệ, trừ số tiền khỏi balance.
4. `show_balance(self)`
- Trả về hoặc hiển thị số dư hiện tại của tài khoản.
5. `display(self)`
- Hiển thị số tài khoản, tên chủ tài khoản và số dư hiện tại.

Bài 7. Lớp Phân số

Xây dựng lớp Fraction.

Thuộc tính:

- `numerator`: tử số.
- `denominator`: mẫu số.

Các phương thức:

1. `__init__(self, numerator, denominator)`
 - Khởi tạo một phân số.
2. `input_fraction(self)`
 - Nhập tử số và mẫu số từ bàn phím.
 - Tiếp tục yêu cầu nhập lại nếu mẫu số bằng 0.
3. `simplify(self)`
 - Rút gọn phân số về dạng tối giản.
4. `value(self)`
 - Tính giá trị thực của phân số: thực hiện phép chia `numerator / denominator`.
5. `display(self)`
 - Hiển thị phân số theo dạng tử số/mẫu số.

Bài 8. Thuộc tính lớp và biến đếm đối tượng

Xây dựng lớp Book.

Thuộc tính đối tượng:

- `book_id`: mã sách.
- `title`: tên sách.
- `author`: tác giả.

Biến lớp:

- `book_count`: lưu tổng số đối tượng sách đã được tạo.

Các phương thức:

1. `__init__(self, book_id, title, author)`
 - Khởi tạo thông tin cho một cuốn sách.
 - Mỗi lần phương thức này được gọi, tăng `Book.book_count` lên 1.
2. `display(self)`

- Hiện thị mã sách, tên sách và tác giả.
3. `show_count()`
- Hiện thị tổng số đối tượng Book đã được tạo.
 - Giá trị được lấy từ biến lớp `Book.book_count`.

Bài 9. Lớp Nhiệt độ (Temperature) - Tên file: OLCB_BaiTap09.py

Xây dựng lớp Temperature biểu diễn và chuyển đổi thang đo nhiệt độ.

Yêu cầu:

- Thuộc tính: `celsius` (độ C, kiểu float). Nếu khởi tạo nhỏ hơn -273.15 (độ không tuyệt đối), tự động gán giá trị về -273.15.
- Phương thức:
 - + `__init__(self, celsius=0.0)`: Khởi tạo nhiệt độ.
 - + `to_fahrenheit(self)`: Tính và trả về độ F ($F = C * 1.8 + 32$).
 - + `to_kelvin(self)`: Tính và trả về độ K ($K = C + 273.15$).
 - + `display(self)`: Hiện thị nhiệt độ dưới cả 3 thang đo (C, F, K) làm tròn 2 chữ số thập phân.

B. Phần Nâng cao

Bài 10. Lớp Điểm tọa độ 2D (Point2D) - Tên file: OLCB_BaiTap10.py

Xây dựng lớp Point2D biểu diễn một điểm trong hệ tọa độ Descartes hai chiều.

Yêu cầu:

- Thuộc tính: `x`, `y` (tọa độ trục hoành và trục tung, kiểu float).
- Phương thức:
 - + `__init__(self, x=0.0, y=0.0)`: Khởi tạo tọa độ.
 - + `distance_to(self, other)`: Tính và trả về khoảng cách Euclid từ điểm hiện tại đến đối tượng điểm 'other' truyền vào.
 - + `display(self)`: In tọa độ điểm ra màn hình dưới dạng (x, y).

Bài 11. Hệ thống Quản lý Khách sạn - Tên file: OLCB_BaiTap11.py

Xây dựng lớp HotelRoom gồm các thuộc tính: `room_number`, `room_type` (Standard/Deluxe/Suite), `price_per_night`, `is_available` (mặc định True).

Yêu cầu:

Xây dựng lớp HotelManager quản lý danh sách phòng. Có các phương thức: `add_room()`, `book_room(room_number)` (đổi trạng thái trống), `check_out(room_number, nights)` (tính tiền phòng thực nhận và trả phòng), `find_available_rooms()` hiển thị các phòng trống.

Bài 12. Quản lý Thư viện Sách - Tên file: OLCB_BaiTap12.py

Xây dựng lớp LibraryBook gồm: `book_id`, `title`, `author`, `is_borrowed` (mặc định False).

Xây dựng lớp LibraryManager quản lý danh sách sách. Có các phương thức: `add_book(book)`, `borrow_book(book_id)` (thông báo lỗi nếu sách đã bị mượn),

`return_book(book_id)`, `search_by_title(keyword)` tìm kiếm gần đúng các đầu sách theo từ khóa.

Bài 13. Giao dịch Tài khoản Ngân hàng - Tên file: OLCB_BaiTap13.py

Xây dựng lớp `BankAccount` (kế thừa từ lớp cơ bản trước đó) bổ sung thêm thuộc tính `transaction_history` (kiểu `list`). Xây dựng lớp `BankSystem` quản lý danh sách tài khoản, có phương thức: `transfer(from_acc_num, to_acc_num, amount)` thực hiện chuyển khoản an toàn giữa hai chủ tài khoản, và hiển thị lịch sử giao dịch chi tiết của từng người bao gồm thời gian và số dư biến động.

Bài 14. Quản lý Đơn hàng bán lẻ - Tên file: OLCB_BaiTap14.py

Sử dụng lớp `Product` ở Bài tập 5. Xây dựng lớp `OrderItem` chứa đối tượng `Product` và `quantity`. Lớp `Order` gồm mã đơn hàng `order_id`, danh sách các `OrderItem` và mã giảm giá `voucher_discount` (phần trăm giảm từ 0% đến 50%). Có phương thức: `add_item()`, `calculate_total()` tính tổng tiền sau khi áp voucher, `display_order()` in chi tiết đơn hàng dạng hóa đơn bán lẻ chuyên nghiệp.

Bài 15. Quản lý Điểm Học sinh - Tên file: OLCB_BaiTap15.py

Xây dựng lớp `StudentGrades` gồm: `student_id`, `name`, `grades` (là dictionary lưu điểm các môn: 'Math', 'Literature', 'English').

Xây dựng lớp `GradeBook` chứa danh sách học sinh, có phương thức: `add_student()`, `calculate_average(student_id)` tính điểm trung bình ($\text{Toán} \times 2 + \text{Văn} \times 2 + \text{Anh} \times 1$) / 5, `find_top_student()` tìm học sinh có điểm trung bình cao nhất lớp, `display_report_card()` in phiếu điểm lớp học sắp xếp theo điểm trung bình giảm dần.

Bài 16. Hệ thống Đặt chỗ Chuyến bay - Tên file: OLCB_BaiTap16.py

Xây dựng lớp `Flight` gồm: `flight_code`, `destination`, `total_seats`, `booked_seats` (mảng lưu danh sách tên khách hàng đã đặt ghế). Phương thức: `book_seat(passenger_name)` (thêm khách hàng, trả về False nếu đã hết ghế), `cancel_seat(passenger_name)` (hủy đặt chỗ), `is_full()` kiểm tra trạng thái đầy tải.

Viết chương trình chính quản lý 3 chuyến bay đồng thời.

Bài 17. Đăng ký Học phần Đại học - Tên file: OLCB_BaiTap17.py

Xây dựng lớp `Course` gồm: `course_code`, `course_name`, `max_students`, `registered_students` (danh sách lưu đối tượng `Student` từ Bài 3).

Xây dựng lớp `CourseRegistration` quản lý quá trình đăng ký: `register_course(student, course)` kiểm tra xem khóa học đã đầy chưa, sinh viên có đủ điều kiện GPA đầu vào không (nếu có yêu cầu tối thiểu GPA ≥ 5.0), hiển thị danh sách lớp học của từng học phần.

Bài 18. Quản lý Sự kiện và Vé - Tên file: OLCB_BaiTap18.py

Xây dựng lớp Event gồm: event_id, name, date, ticket_price, total_tickets, sold_tickets. Xây dựng lớp EventOrganizer quản lý các sự kiện. Phương thức: sell_ticket(event_id, quantity) thực hiện bán vé, thông báo số lượng vé còn lại; calculate_event_revenue(event_id) trả về doanh thu thực tế; display_active_events() in danh sách sự kiện kèm tỷ lệ lấp đầy ghế.

Bài 19. Giải và Biện luận Phương trình Bậc hai - Tên file: OLCB_BaiTap19.py

Xây dựng lớp QuadraticEquation biểu diễn phương trình $ax^2 + bx + c = 0$.

Yêu cầu: Thuộc tính private __a, __b, __c. Phương thức khởi tạo kiểm tra hệ số __a phải khác 0 (nếu bằng 0 yêu cầu nhập lại hoặc nâng lỗi ValueError). Phương thức get_discriminant() tính delta. Phương thức solve() biện luận và trả về danh sách các nghiệm thực tế của phương trình.

Bài 20. Hệ thống Tiếp nhận Ticket Hỗ trợ - Tên file: OLCB_BaiTap20.py

Xây dựng lớp SupportTicket gồm: ticket_id, customer_name, issue, priority (Low/Medium/High), status (mặc định 'Open').

Xây dựng lớp TicketManager quản lý danh sách yêu cầu hỗ trợ. Phương thức: add_ticket(ticket), process_next_ticket() tự động lấy ra và xử lý ticket có độ ưu tiên cao nhất trước ('High' -> 'Medium' -> 'Low'), update_status(ticket_id, new_status), show_pending_tickets() hiển thị các ticket chưa xử lý.

IV. BÀI TẬP VỀ NHÀ

A. Phần Cơ bản

Bài 1. So sánh hai đối tượng sinh viên (Tên file: VNCB_BaiTap01.py)

Cho lớp Student gồm các thuộc tính cơ bản như:

- student_id: mã sinh viên.
- name: họ tên.
- gpa: điểm trung bình.

Bổ sung các phương thức sau:

1. compare_gpa(self, other_student)
 - Nhận vào một đối tượng Student khác.
 - So sánh gpa của đối tượng hiện tại với other_student.
 - Thông báo một trong ba trường hợp:
 - Sinh viên hiện tại có điểm trung bình cao hơn.
 - Sinh viên được truyền vào có điểm trung bình cao hơn.
 - Hai sinh viên có điểm trung bình bằng nhau.
2. __eq__(self, other)
 - So sánh dữ liệu của hai đối tượng sinh viên.
 - Hai sinh viên được xem là bằng nhau nếu các thuộc tính student_id, name và gpa giống nhau.

Trong chương trình chính, tạo hai đối tượng student1 và student2, sau đó thực hiện:

```
student1.compare_gpa(student2)
print(student1 is student2)
print(student1 == student2)
```

Bài 2. (Tên file: VNCB_BaiTap02.py)

Xây dựng lớp Person.

Phương thức:

1. `__init__(self, full_name, age)`

- Khởi tạo họ tên và tuổi của một người.

Ngoài các phương thức của lớp, sinh viên sử dụng:

- `hasattr(person, "address")`
 - Kiểm tra đối tượng có thuộc tính address hay chưa.
- `setattr(person, "address", value)`
 - Tạo mới hoặc thay đổi giá trị thuộc tính address.
- `getattr(person, "address")`
 - Lấy giá trị đang lưu trong thuộc tính address.
- `delattr(person, "address")`
 - Xóa thuộc tính address khỏi đối tượng.
- `person.__dict__`
 - Quan sát các thuộc tính hiện đang được lưu trong đối tượng.
- `person.__class__.__name__`
 - Xác định tên lớp mà đối tượng thuộc về.
- `dir(person)`
 - Liệt kê các thuộc tính và phương thức mà đối tượng có thể sử dụng.

Bài 3. (Tên file: VNCB_BaiTap03.py)

Xây dựng lớp Triangle.

Các phương thức:

1. `__init__(self, a, b, c)`

- Khởi tạo ba cạnh của tam giác.

2. `is_valid(self)`

- Kiểm tra ba cạnh có tạo thành một tam giác hay không.
- Các cạnh phải lớn hơn 0 và tổng hai cạnh bất kỳ phải lớn hơn cạnh còn lại.

3. `perimeter(self)`

- Tính tổng ba cạnh.
- Trả về chu vi tam giác.

4. `area(self)`

- Tính diện tích theo công thức Heron.
- Chỉ thực hiện khi ba cạnh tạo thành tam giác hợp lệ.

5. `triangle_type(self)`
 - Phân loại tam giác.
 - Cho biết tam giác đều, cân, vuông hoặc thường.
6. `display(self)`
 - Hiển thị ba cạnh, loại tam giác, chu vi và diện tích.

Bài 4. (Tên file: VNCB_BaiTap04.py)

Xây dựng lớp Date.

Các phương thức:

1. `__init__(self, day, month, year)`
 - Khởi tạo ngày, tháng và năm.
2. `is_leap_year(self)`
 - Kiểm tra năm hiện tại có phải năm nhuận hay không.
 - Trả về True hoặc False.
3. `days_in_month(self)`
 - Xác định số ngày tối đa của tháng đang lưu trong đối tượng.
 - Đặc biệt xử lý tháng 2 dựa vào năm nhuận.
4. `is_valid_date(self)`
 - Kiểm tra ngày, tháng, năm có tạo thành một ngày hợp lệ hay không.
5. `display(self)`
 - Hiển thị ngày theo định dạng dd/mm/yyyy.

Bài 5. (Tên file: VNCB_BaiTap05.py)

Xây dựng lớp ElectricBill dùng để biểu diễn và tính toán hóa đơn tiền điện của một khách hàng dựa trên chỉ số điện cũ, chỉ số điện mới và bảng giá điện theo bậc.

Thuộc tính

- `customer_id`: mã khách hàng.
- `customer_name`: họ tên khách hàng.
- `old_index`: chỉ số điện của kỳ trước.
- `new_index`: chỉ số điện của kỳ hiện tại.

Các phương thức

1. `__init__(self, customer_id, customer_name, old_index, new_index)`
 - Khởi tạo một đối tượng hóa đơn điện.
 - Gán mã khách hàng, họ tên khách hàng, chỉ số điện cũ và chỉ số điện mới cho các thuộc tính tương ứng.
2. `consumption(self)`
 - Tính lượng điện khách hàng đã sử dụng trong kỳ.
 - Công thức:

Số điện tiêu thụ = Chỉ số điện mới - Chỉ số điện cũ

- Phương thức trả về số điện tiêu thụ, đơn vị kWh.

3. electricity_cost(self)

- Tính tiền điện trước thuế theo số điện tiêu thụ.
- Áp dụng biểu giá lũy tiến:

Bậc	Mức sử dụng	Đơn giá
1	0–50 kWh	1.984 đồng/kWh
2	51–100 kWh	2.050 đồng/kWh
3	101–200 kWh	2.380 đồng/kWh
4	201–300 kWh	2.998 đồng/kWh
5	301–400 kWh	3.350 đồng/kWh
6	Trên 400 kWh	3.460 đồng/kWh

- Mỗi phần điện được tính theo đơn giá của bậc tương ứng.
- Trả về tổng tiền điện trước thuế.

4. vat(self)

- Tính tiền thuế giá trị gia tăng (VAT).
- Thuế VAT bằng 10% tiền điện trước thuế.
- Công thức:

$$\text{VAT} = \text{Tiền điện trước thuế} \times 10 / 100$$

hoặc:

$$\text{VAT} = \text{Tiền điện trước thuế} \times 0,1$$

5. total_payment(self)

- Tính tổng số tiền khách hàng phải thanh toán.
- Tổng tiền bằng tiền điện trước thuế cộng với tiền VAT.
- Công thức:

$$\text{Tổng tiền thanh toán} = \text{Tiền điện trước thuế} + \text{VAT}$$

6. display_bill(self)

Hiển thị đầy đủ thông tin hóa đơn điện của khách hàng, gồm:

- Mã khách hàng.
- Họ tên khách hàng.
- Chỉ số điện cũ.
- Chỉ số điện mới.
- Số điện tiêu thụ.
- Tiền điện trước thuế.
- Tiền VAT.
- Tổng số tiền phải thanh toán.

Yêu cầu chương trình chính

- Tạo một đối tượng thuộc lớp ElectricBill.
- Tính số điện tiêu thụ.
- Tính tiền điện trước thuế.
- Tính tiền VAT.

- Tính tổng số tiền phải thanh toán.
- Gọi phương thức `display_bill()` để hiển thị đầy đủ thông tin hóa đơn.

Bài 6. Quản lý danh sách sản phẩm (Tên file: VN_CB_BaiTap06.py)

Xây dựng lớp `ProductManager` dùng để quản lý một danh sách các đối tượng `Product`. Mỗi sản phẩm trong danh sách được phân biệt thông qua mã sản phẩm.

Thuộc tính

- `products`: danh sách dùng để lưu trữ các đối tượng `Product`.

Các phương thức

1. `__init__(self)`

- Khởi tạo đối tượng quản lý sản phẩm.
- Ban đầu, danh sách `products` là một danh sách rỗng.

2. `add_product(self, product)`

- Nhận vào một đối tượng `Product`.
- Kiểm tra mã của sản phẩm cần thêm đã tồn tại trong danh sách hay chưa.
- Nếu mã sản phẩm chưa tồn tại, thêm đối tượng vào danh sách.
- Nếu mã sản phẩm đã tồn tại, thông báo không thể thêm sản phẩm trùng mã.

3. `display_products(self)`

- Duyệt qua toàn bộ danh sách `products`.
- Hiển thị thông tin của từng sản phẩm.
- Nếu danh sách chưa có sản phẩm, thông báo danh sách rỗng.

4. `find_by_id(self, product_id)`

- Nhận vào mã sản phẩm cần tìm.
- Duyệt danh sách để tìm sản phẩm có mã tương ứng.
- Nếu tìm thấy, trả về đối tượng `Product`.
- Nếu không tìm thấy, thông báo sản phẩm không tồn tại.

5. `update_price(self, product_id, new_price)`

- Nhận vào mã sản phẩm và đơn giá mới.
- Tìm sản phẩm có mã tương ứng trong danh sách.
- Nếu tìm thấy, cập nhật lại đơn giá của sản phẩm bằng `new_price`.
- Nếu không tìm thấy, thông báo sản phẩm không tồn tại.

6. `delete_product(self, product_id)`

- Nhận vào mã sản phẩm cần xóa.
- Tìm sản phẩm có mã tương ứng trong danh sách.
- Nếu tìm thấy, xóa sản phẩm khỏi danh sách.
- Nếu không tìm thấy, thông báo sản phẩm không tồn tại.

7. `sort_by_amount(self)`

- Sắp xếp các sản phẩm trong danh sách theo thành tiền giảm dần.
- Thành tiền của mỗi sản phẩm được xác định dựa trên thông tin của đối tượng `Product`.

- Sau khi sắp xếp, sản phẩm có thành tiền lớn nhất đứng đầu danh sách.

Yêu cầu chương trình chính

- Tạo một đối tượng thuộc lớp ProductManager.
- Tạo một số đối tượng Product và thêm vào danh sách.
- Hiện thị danh sách sản phẩm.
- Thực hiện tìm kiếm sản phẩm theo mã.
- Cập nhật đơn giá của một sản phẩm.
- Xóa một sản phẩm khỏi danh sách.
- Sắp xếp danh sách theo thành tiền từ cao xuống thấp và hiện thị kết quả.

Bài 7. (Tên file: VNCB_BaiTap07.py)

Xây dựng lớp Cart dùng để quản lý các sản phẩm được khách hàng thêm vào giỏ hàng.

Thuộc tính

- products: danh sách chứa các đối tượng Product có trong giỏ hàng.

Các phương thức

1. __init__(self)

- Khởi tạo một giỏ hàng rỗng.
- Ban đầu, danh sách products chưa chứa sản phẩm nào.

2. add_product(self, product)

- Nhận vào một đối tượng Product.
- Thêm sản phẩm vào giỏ hàng.
- Nếu sản phẩm đã tồn tại trong giỏ, có thể tăng số lượng thay vì thêm mới một đối tượng khác.

3. remove_product(self, product_id)

- Nhận vào mã sản phẩm cần xóa.
- Tìm sản phẩm có mã tương ứng trong giỏ hàng.
- Nếu tìm thấy, xóa sản phẩm khỏi giỏ.
- Nếu không tìm thấy, thông báo sản phẩm không tồn tại.

4. update_quantity(self, product_id, quantity)

- Nhận vào mã sản phẩm và số lượng mới.
- Tìm sản phẩm có mã tương ứng trong giỏ hàng.
- Nếu tìm thấy, cập nhật lại số lượng.
- Nếu sản phẩm không tồn tại hoặc số lượng không hợp lệ, hiện thị thông báo thích hợp.

5. subtotal(self)

- Tính tổng tiền của tất cả sản phẩm trong giỏ trước khi giảm giá.
- Thành tiền của mỗi sản phẩm được tính bằng:

Thành tiền = Đơn giá × Số lượng

- Tổng tiền giỏ hàng là tổng thành tiền của tất cả sản phẩm.
- Phương thức trả về tổng tiền trước giảm giá.

6. discount(self)

- Tính số tiền được giảm dựa trên tổng giá trị giỏ hàng.
- Áp dụng mức giảm như sau:
 - Tổng tiền dưới 2.000.000 đồng: không giảm.
 - Tổng tiền từ 2.000.000 đồng đến dưới 5.000.000 đồng: giảm 5%.
 - Tổng tiền từ 5.000.000 đồng trở lên: giảm 10%.

Công thức:

Tiền giảm = Tổng tiền × Tỷ lệ giảm

- Phương thức trả về số tiền được giảm.

7. total_payment(self)

- Tính số tiền cuối cùng khách hàng phải thanh toán sau khi trừ giảm giá.

Công thức:

Số tiền thanh toán = Tổng tiền - Tiền giảm

- Phương thức trả về số tiền khách hàng phải trả.

8. display_cart(self)

- Hiển thị thông tin các sản phẩm có trong giỏ hàng, gồm:
 - Mã sản phẩm.
 - Tên sản phẩm.
 - Đơn giá.
 - Số lượng.
 - Thành tiền.
- Đồng thời hiển thị:
 - Tổng tiền trước giảm giá.
 - Số tiền được giảm.
 - Số tiền cuối cùng phải thanh toán.
- Nếu giỏ hàng rỗng, thông báo giỏ hàng chưa có sản phẩm.

B. Phần Nâng cao

Bài 8. Hệ thống Đặt đồ ăn trực tuyến - Tên file: VNNC_BaiTap08.py

Mô phỏng quy trình đặt món của GrabFood / ShopeeFood.

Yêu cầu cụ thể:

- Lớp FoodItem: thuộc tính item_id, name, price.
- Lớp FoodOrder: chứa danh sách các món ăn chọn lựa (dưới dạng danh sách đối tượng FoodItem và số lượng tương ứng), thuộc tính distance_km đại diện cho khoảng cách giao hàng.
- Phương thức của FoodOrder:
 - + calculate_delivery_fee(): Tính phí giao hàng lũy tiến dựa trên khoảng cách:
 - * Từ 0km đến dưới 3km: 15.000 đồng.
 - * Từ 3km đến dưới 6km: 25.000 đồng.
 - * Từ 6km trở lên: 50.000 đồng.
 - + subtotal(): Tính tổng tiền đồ ăn trước phí ship.
 - + total_payment(): Tổng số tiền thanh toán = Tiền đồ ăn + Phí ship.
 - + display_receipt(): In biên lai rõ ràng ra màn hình console sử dụng các dấu phân cách rộng 50 ký tự.

Bài 9. Quản lý Nhân sự Phân cấp Công ty - Tên file: VNNC_BaiTap09.py

Mô phỏng sơ đồ tổ chức phòng ban doanh nghiệp.

Yêu cầu cụ thể:

- Lớp Employee: thuộc tính employee_id, name, position, salary_coefficient (hệ số lương), base_salary (mức lương cơ sở chung của công ty, mặc định là 2.000.000đ). Phương thức calculate_income() trả về Hệ số lương x Lương cơ sở.

- Lớp Manager (kế thừa từ Employee hoặc quản lý độc lập): Ngoài các thuộc tính như Employee, bổ sung danh sách sub_employees (lưu trữ các nhân viên cấp dưới trực thuộc quản lý) và allowance (phụ cấp trách nhiệm). Phương thức calculate_income() trả về Hệ số lương x Lương cơ sở + Phụ cấp.

- Lớp CompanyManager: Quản lý toàn bộ danh sách nhân viên và quản lý trong công ty. Phương thức: add_member(), get_total_payroll() tính tổng quỹ lương công ty, display_organization_tree() hiển thị sơ đồ tổ chức phân cấp rõ ràng.

Bài 10. Hệ thống Đặt vé Rạp chiếu phim - Tên file: VNNC_BaiTap10.py

Xây dựng hệ thống quản lý sơ đồ ghế ngồi phòng chiếu.

Yêu cầu cụ thể:

- Lớp Showtime: thuộc tính show_id, movie_name, time, ticket_price.

- Lớp CinemaRoom: thuộc tính room_name, rows (số hàng ghế), cols (số cột ghế), seat_matrix (ma trận 2 chiều kích thước rows x cols khởi tạo giá trị 0 đại diện cho ghế trống, giá trị 1 đại diện cho ghế đã được đặt thành công).

- Phương thức của CinemaRoom:

- + display_seats(): Hiển thị sơ đồ phòng chiếu (ví dụ các hàng A, B, C... và ghế trống dạng [O], ghế đã đặt dạng [X]).

- + book_seat(row_idx, col_idx): Thực hiện đặt ghế tại vị trí chỉ định. Kiểm tra hợp lệ (chỉ số nằm trong ma trận, ghế chưa bị đặt). Nếu đặt thành công chuyển trạng thái sang 1.

- + calculate_revenue(showtime): Tính tổng doanh thu dựa trên số ghế đã đặt x giá vé của Showtime.

- + display_ticket_receipt(row_idx, col_idx, showtime): Xuất thông tin hóa đơn vé điện tử chi tiết.

-- Hết --